ASSIGNMENT-9.1
2303A52249
A.VAMSHI KRISHNA

TASK-1:

Consider the following Python function: def find_max(numbers): return max(numbers)

Prompt: Write documentation for the function in all three formats:

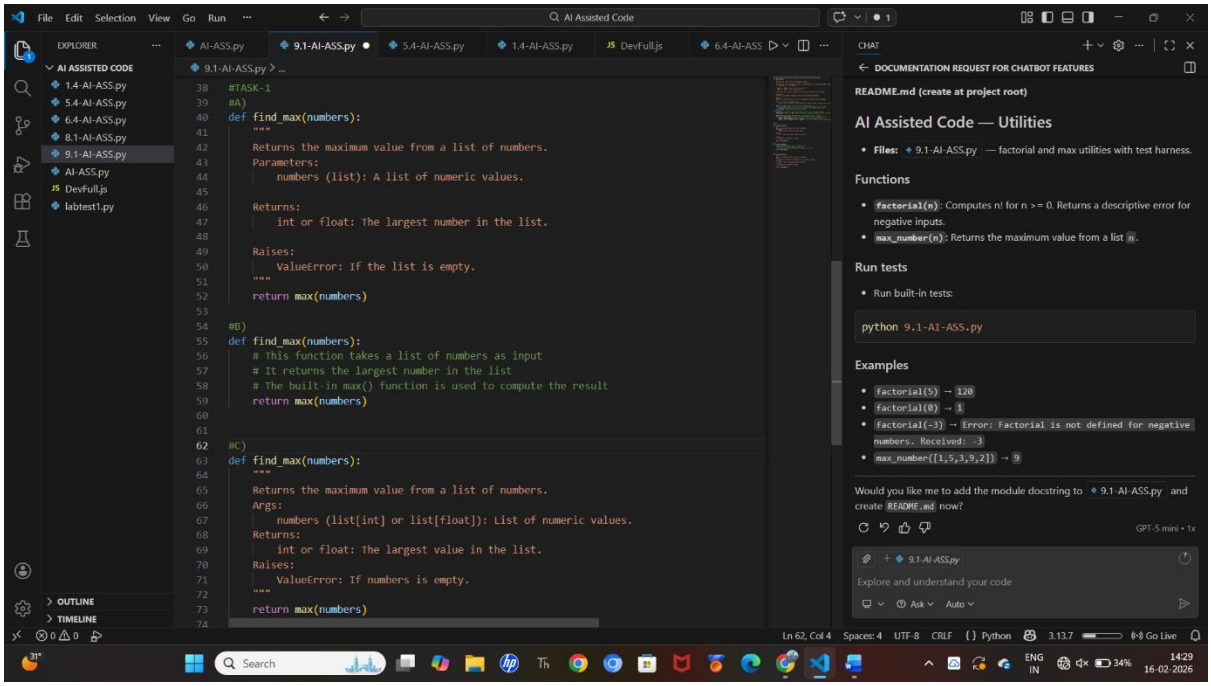
(a) Docstring

(b) Inline comments

(c) Google-style documentation

• Critically compare the three approaches. Discuss the advantages, disadvantages, and suitable use cases of each style.

• Recommend which documentation style is most effective for a mathematical utilities library and justify your answer.



1. Docstring (Standard Style):

Advantages

- Accessible via help(find_max)
- Good for small projects
- Follows Python conventions (PEP 257)

Disadvantages

- Less structured than Google style
- Not always consistent formatting

Suitable Use Case

- Small academic projects
- Simple scripts
- Beginner-level coding

2️⃣ Inline Comments:

Advantages

- Easy to write
- Helpful for explaining logic line-by-line
- Good for beginners

Disadvantages

- Not accessible via documentation tools
- Can become messy in large programs
- Harder to maintain

Suitable Use Case

- Explaining complex logic
- Debugging
- Quick clarification inside code

3️⃣ Google-Style Documentation:

Advantages

- Clear structured format (Args, Returns, Raises)
- Used in large-scale projects
- Compatible with tools like Sphinx
- Easy to read and maintain

Disadvantages

- Slightly longer to write
- Requires formatting discipline

Suitable Use Case

- Professional software development
- Libraries and APIs
- Team projects

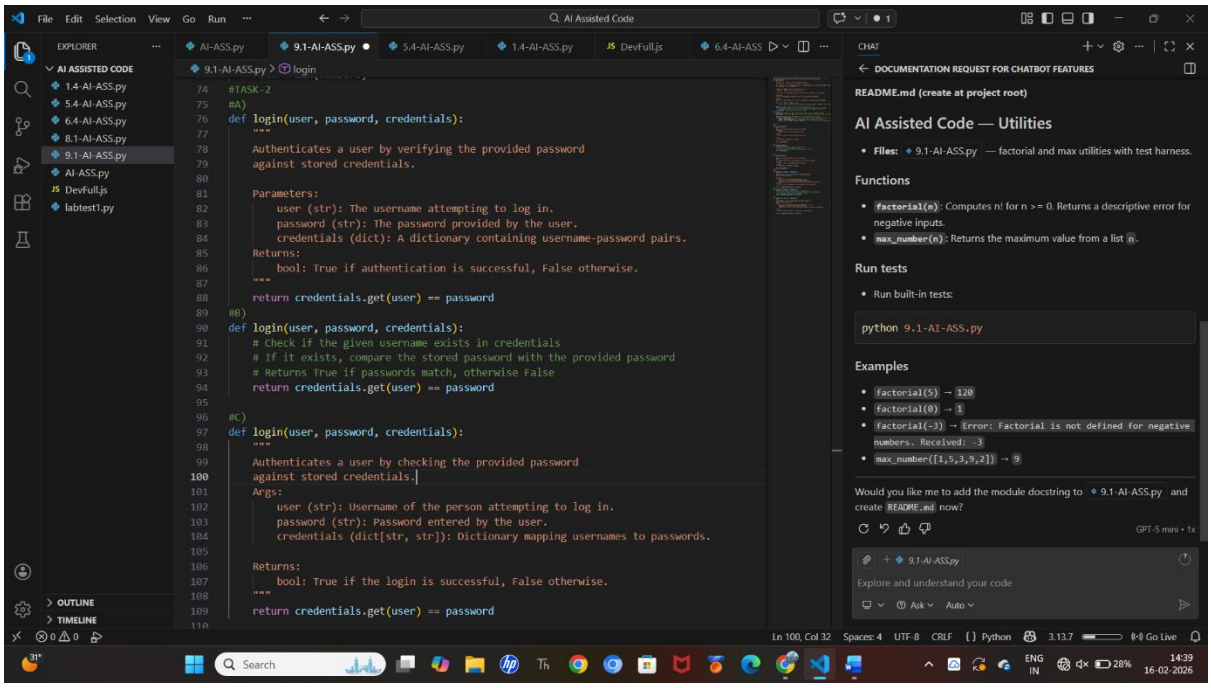
TASK-2:

Prompt: Consider the following Python function:

def login(user, password, credentials):
return credentials.get(user) == password

Task:
1. Write documentation in all three formats.
2. Critically compare the approaches.
3. Recommend which style would be most helpful for new developers onboarding a project, and justify your choice.

(a) Standard Docstring
(b) Inline Comments
(c) Google-Style Docstring



1. Standard Docstring:

Advantages

- Clean and simple
- Easily accessible using help(login)
- Suitable for small applications

Disadvantages

- Formatting may vary between developers
- Slightly less structured than Google style

2️⃣ Inline Comments:

Advantages

- Very easy to understand for beginners
- Good for explaining logic inside complex code

Disadvantages

- Cannot be extracted by documentation tools
- Not suitable for large projects
- Does not describe parameters clearly

3️⃣ Google-Style Docstring:

Advantages

- Clearly separates Args and Returns
- Industry-standard format
- Easy for IDEs and documentation generators
- Very readable for teams

Disadvantages

- Slightly longer to write
- Requires consistent formatting

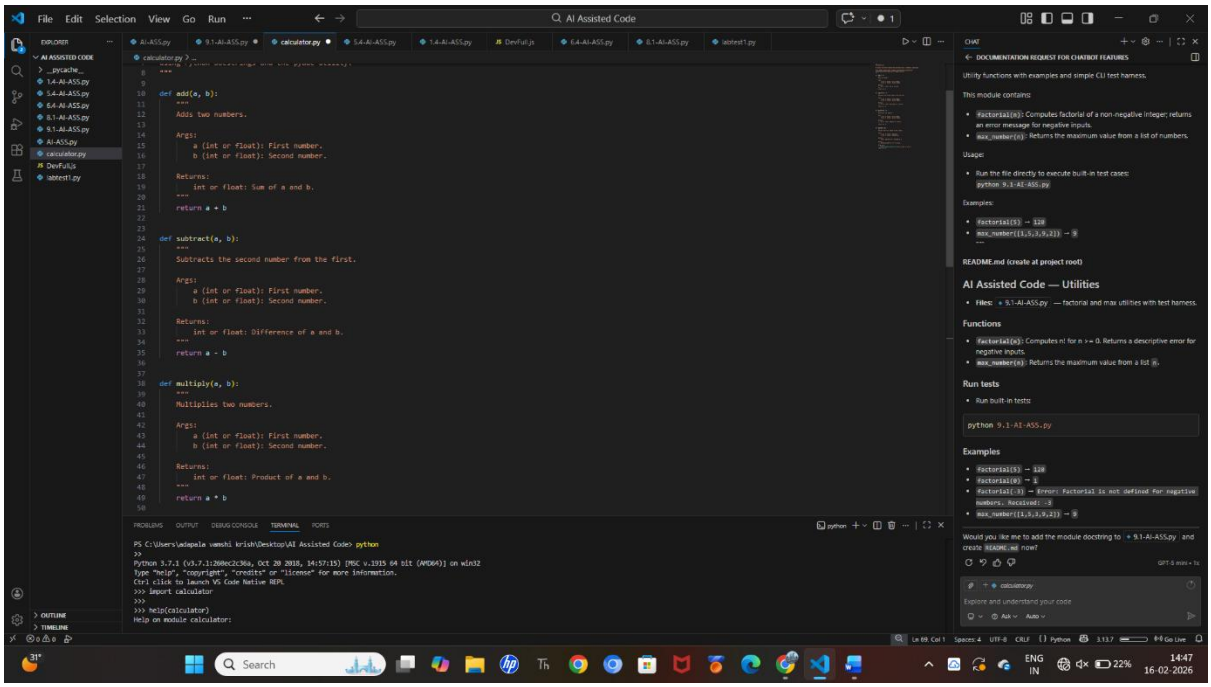
TASK-3:

Prompt: Calculator (Automatic Documentation Generation)

Task: Design a Python module named calculator.py and demonstrate automatic documentation generation.

Instructions:
1. Create a Python module calculator.py that includes the
following functions, each written with appropriate docstrings:
o add(a, b) – returns the sum of two numbers
o subtract(a, b) – returns the difference of two numbers
o multiply(a, b) – returns the product of two numbers
o divide(a, b) – returns the quotient of two numbers
2. Display the module documentation in the terminal using Python's documentation tools.
3. Generate and export the module documentation in HTML format using the pydoc utility, and open the generated HTML file in a web browser to verify the output.

CODE:



Observation:

During this experiment, a Python module named calculator.py was created with properly written docstrings for each function. The module documentation and function-level documentation were automatically generated using Python's built-in documentation tool (pydoc).

TASK-4:

Prompt: Conversion Utilities Module

Task:

1. Write a module named conversion.py with functions:

o decimal_to_binary(n)

o binary_to_decimal(b)

o decimal_to_hexadecimal(n)

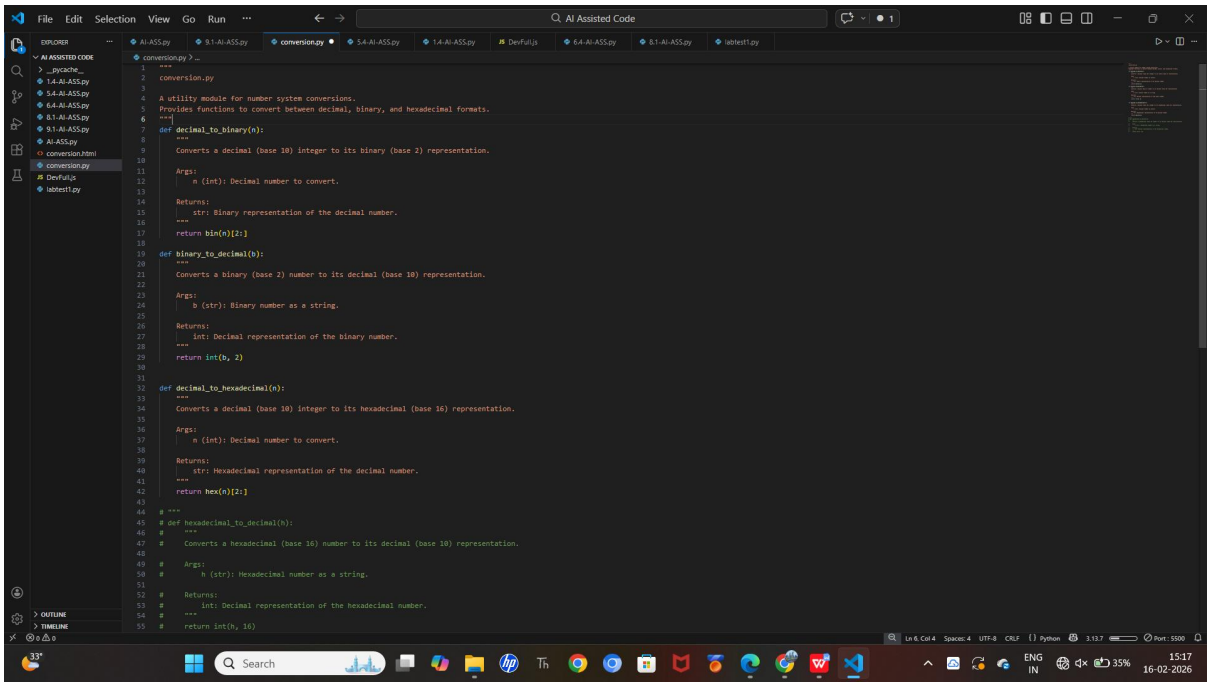
2. Use Copilot for auto-generating docstrings.

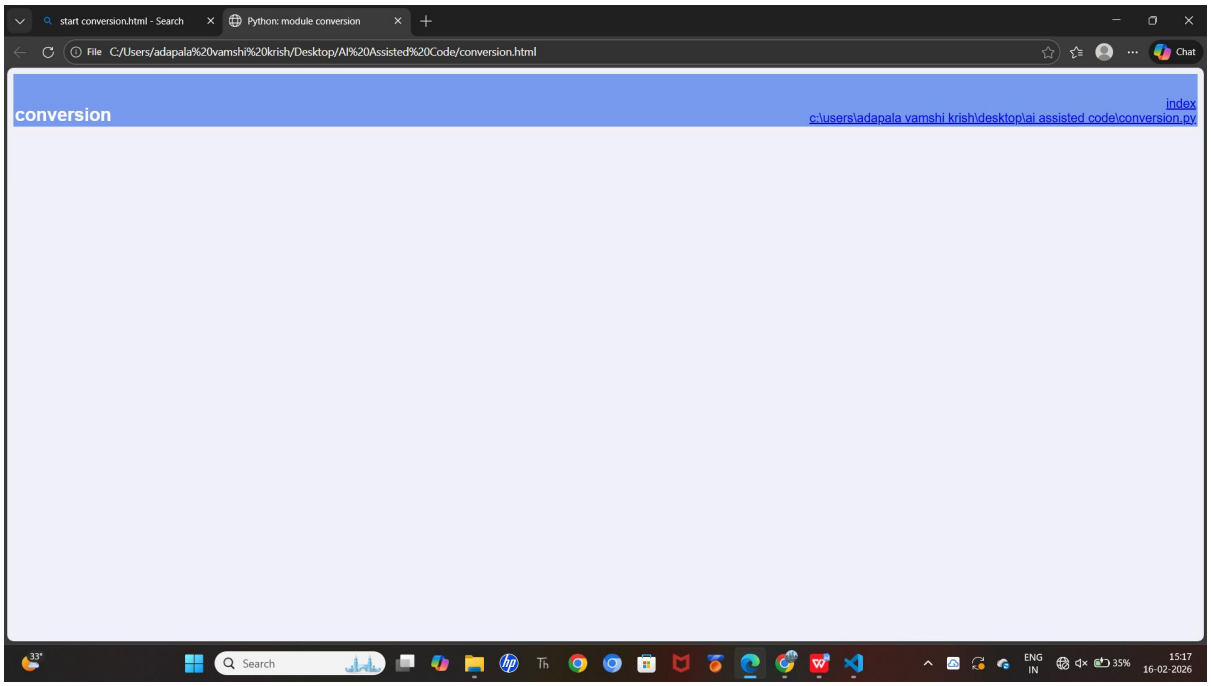
3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a

browser.

CODE:





Observation :

The conversion.py module successfully demonstrated automatic documentation generation using Python's pydoc tool. The AI-assisted docstrings improved clarity and structure. The documentation was easily exported to HTML format, making it suitable for professional use and sharing.

TASK-5:

Prompt: Course Management Module

Task:

1. Create a module course.py with functions:

o add_course(course_id, name, credits)

o remove_course(course_id)

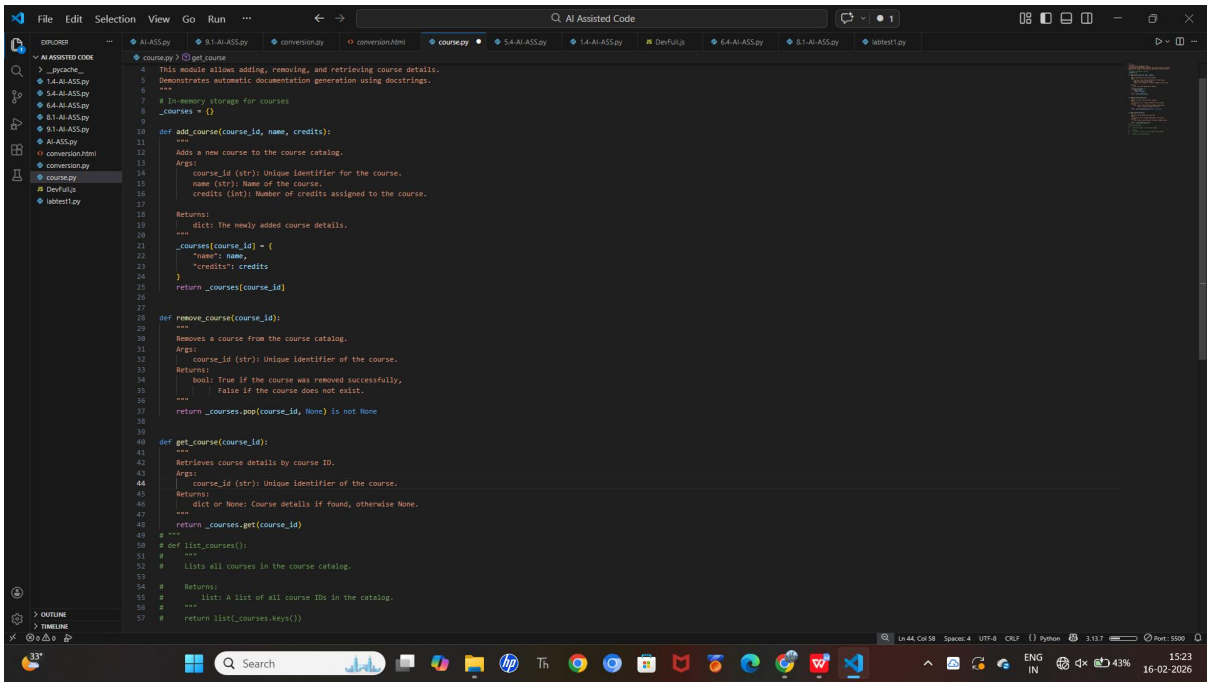
o get_course(course_id)

2. Add docstrings with Copilot.

3. Generate documentation in the terminal.

4. Export the documentation in HTML format and open it in a browser.

CODE:



Observation:

The course.py module successfully demonstrated automatic documentation generation. The docstrings written using Copilot were extracted by pydoc and displayed both in the terminal and HTML format. This experiment highlights the importance of structured documentation for maintainability and onboarding new developers.